Versión 1.1

# Paradigmas de Programación

## Introducción

ING. DAVID CECCHI
ING. JACOBO DOMINGUEZ
ING. NICOLÁS TIZZANO

UNIVERSIDAD TECNOLOGICA NACIONAL
Facultad Regional San Nicolás
Ingeniería en Sistemas

# Breve reseña histórica

En 1928 David Hilbert planteó un desafío denominado “El problema de la decisión” (Entscheidungsproblem). Este problema buscaba responder una pregunta fundamental en lógica matemática:
**¿Existe un algoritmo universal y automático que, dada cualquier fórmula lógica de primer orden, decida en un tiempo finito, si esa fórmula es válida?**

***Ejemplificando (elaborado con colaboración de IA)***

***Ejemplo 1:***
***Fórmula (enunciado) a evaluar:***
*(∀x (Mamífero(x) → Vertebrado(x)) ∧ ∀x (Vertebrado(x) → TieneCélulas(x))) → ∀x (Mamífero(x) → TieneCélulas(x))*

***En palabras**: "Si todo mamífero es vertebrado, y todo vertebrado tiene células, entonces todo mamífero tiene células".*

***¿Es válida?: SÍ** - Es lógicamente válida por transitividad.*

***Importante:** La validez NO depende de que la proposición sea biológicamente cierta. **Es válida porque la estructura lógica es correcta**.*

***Ejemplo 2:***
***Fórmula (enunciado) a evaluar:***
*(∀x (Humano(x) → Mortal(x)) ∧ ∀x (Mortal(x) → Griego(x))) → ∀x (Humano(x) → Griego(x))*

***En palabras**: "Si todo humano es mortal, y todo mortal es griego, entonces todo humano es griego".*

***¿Es válida?: Si** - También sería lógicamente válido, pero se debe tener presente que validez ≠ verdad factual, incluso podría presentarse una proposición válida que plantee un absurdo.*

***Ejemplo 3:***
***Fórmula (enunciado) a evaluar:***
(∀x (Estudiante(x) → Inteligente(x)) ∧ ¬Estudiante(María)) → ¬Inteligente(María)

***En palabras**: "Si todos los estudiantes son inteligentes, y María no es estudiante, entonces María no es inteligente".*

***¿Es válida?: No** – a partir de las premisas que componen la proposición es imposible inferir la inteligencia de María.*

***Contraejemplo:***
*Dominio: {María, Juan, Andrea}*
*Estudiante(Juan) = Verdadero*
*Estudiante(Andrea) = Verdadero*
*Estudiante(María) = Falso*
*Inteligente(Juan) = Verdadero*
*Inteligente(Andrea) = Verdadero*
*Inteligente(María) = Verdadero*

En 1936 Turing y Church demostraron, de forma independiente, que aquello planteado por Hilbert no era posible: no existe procedimiento universal con esas características.

- **Turing** lo hizo a través del modelo de Máquina de Turing (una máquina teórica que podía realizar cualquier calculo computable), con el "problema de la detención", demostrando que es imposible construir un algoritmo que pueda determinar si cualquier otro algoritmo eventualmente se detendrá o entrará en un bucle infinito. Razonamiento por proceso de reducción:
  **Hipótesis:** Suponemos que el Entscheidungsproblem es decidible.
  **Construcción:** Usamos ese supuesto algoritmo para resolver la Detención.
  **Contradicción:** Pero sabemos que desde la lógica la Detención es indecidible.
  **Conclusión:** El Entscheidungsproblem resulta ser indecidible.

  ***El Problema de la Detención***

  *Supongamos que tenemos una función mágica booleana Analizadora(programa, entrada), la cual retornará verdadero si el programa analizado finaliza y retornará falso si cae en un bucle infinito.*
  *Creamos un nuevo programa (perverso) que llama a esa función, y su lógica sería la siguiente:*
  1. *Si Analizadora dice que el programa va a parar, entonces entra en un bucle infinito adrede.*
  2. *Si Analizadora dice que el programa no va a parar, entonces termina inmediatamente (return).*

  *Contradicción (Dilema Lógico):*
  *Cuando ejecutamos el programa perverso sobre sí mismo, el sistema colapsa lógicamente.*
  1. *Si Analizadora dice que "para", el programa perverso "buclea".*
  2. *Si Analizadora dice que "buclea", el programa perverso "para".*

  ***La Paradoja del Barbero***

  *Analogía con el Problema de la Detención: cuando la autoreferencia es el núcleo del problema.*
  *El regente de un pueblo, caracterizado por su obsesión con el cuidado de la imagen e higiene, decide hacer extensiva sus costumbres a todo el pueblo y dicta una nueva ley:*
  *"Todos en el pueblo deben estar afeitados, será responsabilidad del barbero afeitar a todas aquellas personas que no se afeiten a sí mismas, y solo a ellas".*
  *Pero el pequeño pueblo cuenta con un único barbero y surge una pregunta fatal: ¿Quién afeita al barbero?*

  *Contradicción (Dilema Lógico):*
  1. *Si el barbero se afeita a sí mismo: entonces rompe la regla, porque la ley dice que él solo afeita a quienes no se afeitan a sí mismos. Por lo tanto, no debería afeitarse.*
  2. *Si el barbero NO se afeita a sí mismo: Entonces él entra en el grupo de personas que "no se afeitan a sí mismas". Pero la ley dice que el barbero debe afeitar a ese grupo y, por lo tanto, debería afeitarse.*

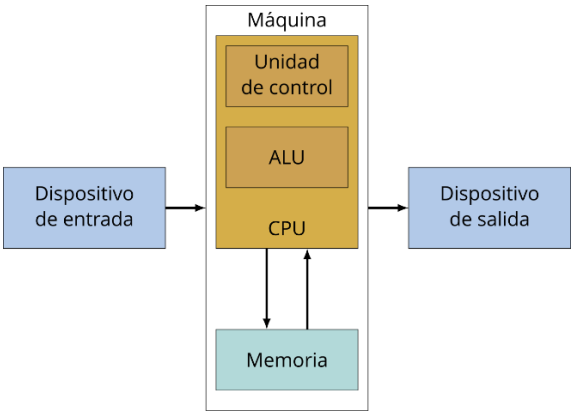
- **Church** lo hizo con su Cálculo Lambda (base matemática para el paradigma funcional), mostrando que no existe un procedimiento efectivo para determinar si dos expresiones lambda son equivalentes (producirán el mismo resultado) o si una expresión lambda terminará su reducción (alcanzará un resultado) o entrará en un bucle infinito de reducción.

Increíblemente, pese a partir de enfoques totalmente diferentes, ambos demostraron matemáticamente que sus modelos (Máquina de Turing y Cálculo Lambda) son equivalentes en poder computacional: cualquier función computable en uno puede computarse en el otro.
Sin embargo, a partir de sus demostraciones fue posible llegar a una deducción aún más relevante: entender que *la lógica pura (software) era independiente de la ejecución física (hardware).*
Antes de los planteos formulados por Church y Turing, las "máquinas calculadoras" eran dispositivos físicos específicos, diseñados para realizar cálculos particulares: fueron creadas para dar solución a una problemática preestablecida.

Teniendo presente que la Máquina de Turing era un modelo teórico (abstracto), que de ser creado podría ejecutar diferentes algoritmos simplemente cambiando las instrucciones en la cinta, permitió plantearse la dicotomía entre los conceptos software y hardware.
Por su lado, Church demostró que la lógica computacional es independiente de la implementación física, ya que su Cálculo Lambda era puramente matemático, en ningún momento hacía referencia a engranajes, circuitos o mecanismos. Demostrando que "computar" es un concepto lógico-matemático, no físico.
En consecuencia, sus estudios sirvieron como base para entender que:

- La lógica (**Software**: algoritmo), es independiente de cómo se implemente físicamente. Turing demostró que el "cómputo" es un proceso simbólico y algorítmico. Lo esencial son las reglas (la lógica), no así el elemento físico que las ejecuta.
- El elemento físico (**Hardware**: transistores, circuitos, procesadores) responsable de la ejecución, es el huésped donde la lógica vive temporalmente.

Así fue posible evolucionar, pasar de construir "máquinas de un solo propósito" a construir "máquinas de propósito general".
Esta revolución conceptual sentó las bases teóricas para la computación moderna. En 1945, John von Neumann tradujo estas ideas abstractas en la **arquitectura de programa almacenado**: el diseño que separa físicamente el hardware (procesador, memoria) del software (instrucciones y datos almacenados en memoria). Desde la EDVAC hasta las computadoras actuales, todas son implementaciones físicas de la misma idea abstracta que Turing y Church formalizaron.




*Arquitectura John Von Neumann*

***Máquina de Turing***

*En el contexto de la Teoría de la Computabilidad, una función es un modelo matemático que expresa un problema determinado.*
*Una función es computable si existe una Máquina de Turing (MT) que pueda calcularla (resolverla).*
*Una Máquina de Turing específica, es un modelo matemático (una abstracción formal de un proceso de computo), no una construcción física real, que representa un algoritmo que resuelve una determinada función computable.*
*Turing, utilizó una analogía mecánica para la MT a través de 3 componentes:*

- *Una **cinta infinita** dividida en celdas donde cada una de ellas contiene un símbolo; comúnmente 0, 1 y blancos.*
- *Un **cabezal** capaz de leer la celda actual, escribir sobre la misma (si es necesario) y desplazarse hacia la izquierda o derecha de dicha celda.*
- *Una **tabla de reglas** que definen la acción que debe tomar la máquina dependiendo de su estado interno y el símbolo que acaba de ser leído.*

*Turing llevó la computación humana a sus elementos más primitivos y mecánicos (leer un símbolo, escribir un símbolo, moverse a la izquierda o derecha, cambiar de estado mental) y demostró que cualquier procedimiento algorítmico que un humano pueda ejecutar mecánicamente (despojado de cualquier rastro de "intuición" o "inteligencia" humana) puede ser realizado por esta máquina abstracta.*

*Es posible trazar un paralelo entre la analogía mecánica de la máquina de Turing y un computador actual:*

***Cinta**: memoria.*
***Símbolos**: bits/bytes.*
***Cabezal**: ALU (unidad aritmético lógica) del procesador.*
***Estado interno**: registros del procesador.*
***Tabla de reglas**: aplicación.*

***Cálculo Lambda***

*El Cálculo Lambda es un lenguaje matemático para describir procesos a través de tres elementos: variables, funciones anónimas y aplicación de funciones (forma de expresar una función con su argumento). Lo relevante de este sistema radica en su simplicidad: mediante la composición de estas funciones anónimas y un mecanismo llamado β-reducción (que define como se deben evaluar las expresiones) es posible representar cualquier cálculo imaginable, desde simples operaciones aritméticas hasta estructuras de datos complejas y lógica condicional. Este modelo es tan poderoso como cualquier lenguaje de programación moderno, demostrando que las funciones son suficientes para expresar cualquier algoritmo computable.*

## Programación en las primeras computadoras

La programación ha evolucionado considerablemente desde sus orígenes en los años 1940s. **ENIAC** (1945), considerada la primera computadora electrónica de propósito general, se "programaba" mediante la reconfiguración manual de cables y switches, un proceso que podía tomar días para elaborar un solo programa y los datos eran ingresados a través de tarjetas perforadas. Este método primitivo, dio paso a **EDVAC** (1949, operativa en 1951), que implementó la revolucionaria arquitectura de von Neumann con el concepto de programa almacenado, permitiendo a través de la lectura de una cinta perforada que tanto los datos como las instrucciones fueran leídos y se almacenaran en su memoria. Para programarla se utilizaba código de máquina. Paralelamente, **EDSAC** (1949) se convirtió en la primera computadora operativa con programa almacenado, introduciendo el uso del lenguaje ensamblador (versión primitiva) y creando la primera biblioteca de subrutinas documentadas, sentando las bases de la programación modular. Finalmente, **UNIVAC I** (1951), la primera computadora comercialmente disponible, se programaba en lenguaje C-10 (combinaba código de máquina y ensamblador), implementando cintas magnéticas para las operaciones de carga de instrucciones y datos.

**Código de Máquina**: sus instrucciones y datos se expresan en binario, o su representación hexadecimal. Es el idioma que el CPU puede ejecutar directamente sin necesidad de traducción, por ser el lenguaje que entienden los circuitos.
**Lenguaje Ensamblador**: sus instrucciones se encuentran en forma de mnemónicos y las direcciones de memoria se referencian a través de etiquetas, haciendo el código mucho más comprensible para los humanos. Finalizado el código fuente, se requiere de un programa denominado Assembler para traducir dichas instrucciones a código de máquina.

## Paradigmas

Un paradigma plantea un enfoque, refiere a un punto de vista sobre un tema determinado. En consecuencia, cada aspecto de la vida o área de estudio cuenta con paradigmas propios. Ejemplificando, en el área de

metodologías de investigación (materia de años superiores) pueden mencionarse el paradigma positivista, constructivista, falsacionista, entre muchos otros. En programación ocurre una situación similar, existen múltiples paradigmas: estructurado, orientado a objetos, funcional, lógico, etc.
Un paradigma de programación, puede definirse como un conjunto de conceptos, métodos y teorías que guían el proceso de codificación (construcción) de un software, definiendo así su estructura interna. Por lo tanto, un paradigma de programación establece una manera de abordar o analizar una problemática determinada y plantear su resolución desde la programación.
Los paradigmas de programación suelen agruparse en dos grandes categorías: Imperativos y Declarativos.

**Imperativos**: hacen énfasis sobre **cómo** obtener el resultado deseado, detallando la secuencia de instrucciones que deben ejecutarse para alcanzarlo. Recurren a estructuras de control (condicionales, selecciones, iteraciones, etc.) para definir el flujo de ejecución y modificar el estado interno (memoria) del autómata a través de asignaciones a variables.
En este grupo, se encuentra el paradigma estructurado, procedural, orientado a objetos, orientado a eventos, etc.
Cabe aclarar que, si bien la programación orientada a objetos plantea el concepto de encapsulamiento y polimorfismo, los métodos en cuestión internamente detallan las instrucciones necesarias para llevarlos a cabo. Por la razón descripta anteriormente, la POO es considerada dentro de los paradigmas imperativos.

**Declarativos**: hacen énfasis sobre **qué** se desea obtener sin concentrarse en detallar cómo lograrlo.
En lugar de plantear la secuencia de instrucciones que constituyen la solución (algoritmo), se basan en describir el problema, especificando ("declarando") un conjunto de condiciones, proposiciones, restricciones, ecuaciones, etc. Posteriormente, por medio de mecanismos internos de control, el sistema determinará la forma óptima de obtener la solución (teniendo en cuenta las declaraciones realizadas con anterioridad), sin necesidad que el desarrollador haya especificado detalladamente como hallarla.
En este grupo, se encuentra el paradigma funcional, lógico, reactivo, etc.

## Paradigmas Imperativos

**Paradigma Procedimental (1950s)**
El paradigma procedimental plantea el desglose de un programa en bloques reutilizables de código llamados procedimientos, también conocidos como rutinas o subprogramas.
Un procedimiento está constituido por una secuencia de instrucciones que se repite dentro del algoritmo. A través del nombre asignado al procedimiento, el programador puede hacer referencia al mismo sin necesidad de volver a detallar todas las instrucciones que lo componen.

Pros:

- Modularidad y reutilización de código.
- Facilita el mantenimiento al subdividir el código en bloques.

Contras:

- Puede generarse dependencia entre procedimientos, cuando se modifican variables globales, se llaman entre ellos, se modifica la firma de un procedimiento obligando a modificar las líneas que los invocan, etc.
- Falta de mecanismos robustos para implementar abstracción y encapsulamiento.

Algunos lenguajes que permiten aplicar el paradigma: primera versión de Fortran (1957), Cobol (1959), Algol 60 (1960 si bien contaba con estructuras de control aún mantenía el uso del goto), Basic 64 (1964).

**Paradigma Estructurado (1960s)**
Los códigos fuentes generados hasta el momento, aún con los lenguajes de alto nivel que permitían implementar el paradigma procedimental, eran extremadamente extensos donde reinaban los saltos de línea.

Desarrollar, interpretar y testear el código fuente representaba una gran complejidad, miles de líneas una tras otra ("código espagueti") pobladas de saltos (sentencia goto) donde un mínimo error resultaba casi imposible de identificar rápidamente, haciendo que estos tiempos se extendieran incluso por días.
En 1966, Böhm y Jacopini, demostraron a través de su teorema, que todo algoritmo (que resuelve una función computable) sin importar su complejidad, puede construirse recurriendo a tres estructuras de control:

- **Secuencia**: ejecución de una instrucción tras otra en orden secuencial.
- **Selección**: ejecución condicional, es decir, una instrucción u otra dependiendo del valor de una condición.
- **Iteración**: ejecución repetida de un bloque de instrucciones mientras se cumpla una condición determinada.

El "inconveniente" con el teorema radicaba en su enfoque puramente matemático. No aclaraba los beneficios que brindaba su tesis, cuya implementación permitiría obtener un código más limpio y legible, aunque sí podría ser referenciado para aseverar que dicha tarea era posible.
En 1968, Edsger Wybe Dijkstra, publicó un artículo muy importante titulado "GO TO statement considered harmful" que tuvo una gran difusión, allí detallaba las desventajas e inconvenientes de recurrir a la sentencia goto. Planteaba que el uso de dicha sentencia hacía imposible seguir mentalmente el flujo de un programa, consecuentemente, iba en perjuicio de las tareas de desarrollo, mantenimiento y testeo. Además, gracias a Böhm-Jacopino, con el respaldo de su teorema, podía asegurar que el uso de goto no era necesario.
En pocas palabras, había generado un manifiesto sobre la calidad del software.
Dijkstra, con su publicación, llevó a los desarrolladores a adoptar un enfoque más estructurado y disciplinado al momento de codificar. No solo cambió la teoría de la programación, sino que a través de éste cambio influyó en el diseño de los lenguajes futuros (ej.: Pascal-1970, C-1972), donde se incorporaron las estructuras de control planteadas por el teorema.
De esta manera se sentaron las bases de la programación estructurada, permitiendo construir programas complejos bien organizados y mejorando la calidad del código.
Pros:

- Sencillo de aprender.
- Claridad de ejecución (es más intuitivo).
- Se facilitan las tareas de corrección y testeo.

Contras:

- Falta de mecanismos robustos para implementar abstracción y encapsulamiento.

Algunos lenguajes que permiten aplicar el paradigma: Algol 68 (1968), Pascal (1970), C (1972).

**Paradigma Orientado a Objetos (1960s)**
El paradigma orientado a objetos plantea que un sistema puede ser visto (modelado) como un conjunto de entidades denominadas objetos que cuentan con ciertos atributos (propiedades) y un determinado comportamiento que estará definido por las acciones (métodos) que pueden realizar.

***Ejemplificación sobre el modelado de un Auto***

| ***Propiedades:*** | ***Métodos:*** |
|---|---|
| *Color* | *Encender()* |
| *Modelo* | *Avanzar()* |
| *Cantidad de Puertas* | *Retroceder()* |
| *Tiene ABS* | *Frenar()* |
| *Tipo de Tracción* | *Apagar()* |

Los programas desarrollados implementando este paradigma están constituidos por un conjunto de objetos cooperantes (es decir, que interactúan entre sí para realizar una tarea), y ésta cooperación es posible a través del envío de mensajes (invocaciones a sus métodos).

***Conceptos Asociados***

***Objeto:***
*Es una entidad que tiene un conjunto de responsabilidades (métodos), encapsula un estado interno (valores asociados a sus propiedades encapsuladas) y posee identidad. Todo objeto representa una instancia de una clase.*

***Clase:***
*Se puede considerar como el molde para un tipo determinado de objetos. Allí se definen las propiedades y métodos que tendrán los objetos surgidos de ella.*
*Ejemplificando, la clase Auto definiría las propiedades y métodos comunes a todos los autos que se generen en el sistema. Luego, a cada auto se le deberán definir los valores (estado) adecuados para cada propiedad. Es decir, el AutoA en su propiedad Color podría tener asignado el valor Rojo mientras que el AutoB en su propiedad Color podría tener asignado el valor Negro, pero ambos han surgido de la misma clase Auto por lo cual tienen acceso a las mismas propiedades y métodos.*

*Nota: estos conceptos serán profundizados y complementados con otros cuando se aborde formalmente el paradigma en unidades subsiguientes.*

La mayor dificultad radica en saber descomponer, de la mejor manera, el dominio del problema que se intenta resolver, para lograr identificar y diseñar las clases y sus relaciones adecuadamente.
A través de conceptos como encapsulamiento, herencia y polimorfismo es posible modelar el mundo real de manera efectiva.
Es ampliamente utilizado en diversas áreas de desarrollo debido a su capacidad para modelar sistemas.

Pros:

- Reutilización de código, las clases pueden reutilizarse en diferentes módulos dentro del mismo proyecto e incluso en otros proyectos.
- Se facilitan las tareas de mantenimiento, gracias al encapsulamiento los cambios realizados en una clase no afectan a otras; siempre y cuando se mantenga la interfaz.
- Modelado natural, permite representar problemas del mundo real de manera intuitiva, debido a que los objetos pueden tener su correspondencia directa con entidades reales.
- Mecanismos robustos para implementar abstracción y encapsulamiento.

Contras:

- Curva de aprendizaje, requiere comprender conceptos abstractos (herencia, polimorfismo, encapsulamiento, abstracción, etc.).
- Mayor consumo de recursos, memoria.
- Sobreingeniería, puede caerse en la creación excesiva de capas de abstracción agregando complejidad innecesaria al modelo.

Algunos lenguajes que permiten aplicar el paradigma: Simula (1967), Smalltalk (1980 versión consolidada), C++ (1983), Java (1995), C# (2000).

Debe realizarse una aclaración, existen lenguajes orientados a objetos y otros basados en objetos.

**Orientados a Objetos**: permiten implementar todos los conceptos del paradigma ej. Smalltalk donde es posible definir herencia, encapsulamiento y polimorfismo.

**Basados en Objetos**: permiten implementar solo algunos conceptos del paradigma ej. Visual Basic 6 no soportaba herencia, javascript en su versión original no soportaba herencia tradicional sino a través de prototipos.
La versión actual de javascript es considerada orientada a objetos, ya que permite herencia tradicional a través de nuevas instrucciones que se incorporaron con el transcurrir de las actualizaciones. Aunque por debajo, dichas instrucciones la implementen a través de prototipos, es un aspecto técnico interno del lenguaje y no lo define.

**Paradigma orientado a eventos (1980s)**
La programación orientada a eventos es un paradigma donde el flujo del programa está determinado por eventos que ocurren sin posibilidad de predecir cuándo sucederán.
Un evento es un suceso, son acciones del usuario (clicks, movimientos del mouse, pulsaciones de teclas, etc.) o señales de otros programas.
En lugar de seguir una secuencia lineal de instrucciones, el programa permanece “atento” y responde a estos eventos mediante un mecanismo específico, donde se utilizan "manejadores" o "listeners" los cuales contienen el código a ejecutar.
Este enfoque es fundamental en interfaces gráficas, aplicaciones web y sistemas que requieren interactividad en tiempo real, ya que permite crear software reactivo y eficiente que responde dinámicamente a las acciones del entorno sin bloquear la ejecución del programa.

***Descripción simplificada del proceso de detección de un evento y su atención***

1. *El sistema operativo, al detectar el suceso (ej. un click del usuario), carga el mensaje en la cola de mensajes (Message Queue) asociada al proceso en cuestión. La Message Queue describe a los mensajes en un formato universal (ej.: Win32).*
2. *El mecanismo Event Loop (una de las tantas tareas del Runtime Environment), está continuamente analizando su Message Queue. Al detectar un nuevo mensaje, lo retirará de la cola, le dará el formato correspondiente al framework en cuestión (ej.: .Net) y lo colocará en su propia cola de eventos (Event Queue) para su posterior despacho.*
3. *El mecanismo Event Loop, irá tomando de su Event Queue un evento a la vez y buscará los Listeners suscriptos al evento.*
4. *El mecanismo Event Loop despachará (Dispatch) el evento, es decir, se invocarán y ejecutarán los listeners hallados.*

*Nota: cada proceso cuenta con su propia instancia de Runtime Environment, su propia Message Queue (en el SO) y Event Queue (en el Runtime).*

Pros:

- Interactividad y capacidad de respuesta, permite crear aplicaciones altamente interactivas que responden inmediatamente a las acciones del usuario.
- Desacoplamiento, los componentes (ej. botones) que generan eventos no necesitan conocer quiénes los consumirán.
- Eficiencia en el uso de recursos, el programa no necesita estar constantemente verificando condiciones mediante bucles, sino que permanece inactivo hasta que ocurre un evento.

Contras:

- Curva de aprendizaje, para programadores acostumbrados a la programación secuencial, pensar en términos de eventos requiere un cambio de mentalidad.
- Si múltiples eventos ocurren simultáneamente o se procesan de forma concurrente, pueden surgir problemas de sincronización y estados inconsistentes si no se manejan correctamente.

- Complejidad en el flujo de control, el flujo del programa no es lineal ni predecible, característica que podría dificultar seguir la lógica de ejecución.

Algunos lenguajes que permiten aplicar el paradigma: Hypertalk (1987), Visual Basic (1991), Python (1991 incorporando bibliotecas gráficas como Tkinter), javascript (1995), C# (2000 con WindowsForms).

## Paradigmas Declarativos

**Paradigma Reactivo (2010s)**
El paradigma reactivo está enfocado en analizar los flujos de datos asíncronos (streams), ya sean finitos o infinitos, con el fin de responder automáticamente a los cambios que se producen en ellos mediante la propagación de eventos.
Por lo expresado, el paradigma promueve un diseño declarativo donde se especifica qué debe ocurrir cuando los flujos (datos) cambien, en lugar de cómo manejar cada cambio de forma imperativa.
Para lograr este funcionamiento, plantea la existencia de entidades que deben suscribirse (Observadores) a la entidad que interesa observar (Observable). Simplificando el proceso, cuando el Observable detecta un cambio en el flujo de datos, informa de la situación a cada Observador suscripto, de forma tal que cada uno de ellos actúe según su necesidad.
De esta manera, es posible dar una solución eficiente al uso exagerado de recursos que se presentaría al intentar plantear la misma necesidad con otros paradigmas que llevarían a cada entidad interesada a realizar continuas y repetidas consultas al Observable para evaluar si hubo un cambio en el flujo de datos y actuar conforme a la respuesta obtenida.

***Ejemplificación sobre la implementación del paradigma (elaborado con colaboración de IA)***

*La situación a analizar se refiere al ajuste automático de calidad de video en una aplicación de stream (ej.: Youtube o Netflix):*

- *El **Observable** sería el **monitor de condiciones de red y rendimiento** (NetworkQualityObservable), que constantemente evalúa métricas como ancho de banda disponible, latencia, tasa de pérdida de paquetes, buffer disponible del video, y capacidad de procesamiento del dispositivo. Este Observable emite eventos cada vez que detecta un cambio significativo en estos parámetros, generando un objeto con información {bandwidth: 5.2Mbps, bufferHealth: 85%, latency: 45ms, timestamp: ...} que será enviado a cada observador.*

- *Los **Observadores** serían varios componentes de la aplicación que necesitan reaccionar a los cambios planteados:*
  - *El **VideoQualitySelector**: decide qué resolución cargar: 360p, 480p, 720p, 1080p, etc.*
  - *El **BufferManager**: ajusta cuánto video pre-cargar según la estabilidad de la conexión.*
  - *El **UIIndicator**: muestra al usuario el ícono de calidad actual o advertencias de conexión lenta.*
  - *El **AnalyticsLogger**: registra cambios de calidad para estadísticas.*

*Cada observador se suscribe al NetworkQualityObservable mediante un método como por ejemplo networkQuality.subscribe(observer). Cuando el Observable detecta que el ancho de banda ha caído significativamente, notifica automáticamente a todos sus suscriptores llamando al método onNext(newQualityData) de cada uno de ellos. En ese momento, el VideoQualitySelector recibe la notificación y decide cambiar de 1080p a 480p, el BufferManager reduce el buffer de pre-carga para evitar rebuffering, el UIIndicator actualiza el ícono mostrando temporalmente "Calidad reducida", y el AnalyticsLogger registra el evento.*

*Debe destacarse su eficiencia, el Observable empuja (push) la información solo cuando hay cambios relevantes, evitando que cada observador tenga que estar constantemente preguntando "¿cambió la*

*calidad de red?" (polling), lo cual desperdiciaría recursos computacionales y energía, especialmente en dispositivos móviles donde la eficiencia es crítica.*

Pros:

- Rápida velocidad de respuesta.
- Eficiencia en el uso de recursos al procesar múltiples solicitudes de manera simultánea, alto rendimiento frente a altas demandas.
- Mayor adaptabilidad, permite que las aplicaciones puedan adaptarse rápidamente frente a cambios en los flujos de datos.

Contras:

- Alto consumo de memoria, al necesitar buffers para almacenar los flujos de datos que aún no han sido procesados por los suscriptores: los observadores pueden ser más lentos en procesar que la velocidad a la que brinda información el observable o podría ser necesario mantener la información ya brindada para los nuevos suscriptores.
- Curva de aprendizaje, requiere incorporar y dominar diferentes conceptos relacionados a la manipulación de streams, y con frecuencia, obligará al programador a recurrir y profundizar en el uso de herramientas adicionales afines.
- Podrían acumularse retrasos cuando exista un excesivo número de procesos vinculados al flujo de datos.

Algunas librerías y frameworks que permiten aplicar el paradigma: RxJava (Java), RxJS (JavaScript/TypeScript), RxPy (Python), Rx.Net (C#).
Rx hace referencia a Reactive Extensions, librería desarrollada por Microsoft y luego tomada como referencia por decenas de lenguajes, estableciendo un estándar para el manejo de streams de datos en tiempo real.

**Paradigma Funcional (1950s)**

El paradigma funcional sienta sus bases en el Cálculo Lambda, una teoría matemática desarrollada por Alonzo Church en 1930. Church recurrió al Cálculo Lambda en 1936 para resolver el Problema de la Decisión planteado por Hilbert.
Se caracteriza por tratar la computación como la evaluación de funciones matemáticas puras, evitando los cambios de estado mediante la inmutabilidad, y utilizando funciones de orden superior, recursividad y composición de funciones. En consecuencia, el problema a resolver se plantea normalmente como la evaluación de una función basada en las previamente definidas.
El paradigma funcional puro no requiere de instrucciones de asignación ni estructuras de control iterativas ya que la repetición de instrucciones se lleva a cabo a través de funciones recursivas. Sin embargo, muchos lenguajes de programación funcional admiten el uso de variables, asignaciones y bucles, combinando estilos funcionales con imperativos.

Pros:

- Mayor legibilidad y simplicidad de código, reduciéndose la cantidad de errores.
- Facilita la tarea de depuración y las pruebas de unidad se tornan más confiables gracias a trabajar con funciones puras.
- La inmutabilidad facilita la ejecución en paralelo, múltiples núcleos (hilos) pueden acceder a los datos sin temor a generar conflictos (que un hilo corrompa los datos de otro).

Contras:

- Puede resultar menos eficiente en el uso de recursos (CPU y Memoria).
- Curva de aprendizaje, conceptos propios del paradigma pueden resultar difíciles de incorporar para desarrolladores que provienen de otros paradigmas.

Algunos lenguajes que permiten aplicar el paradigma: Lisp (1958 híbrido), Miranda (1985), Erlang (1986 híbrido), Haskell (1990), Scala (2004 híbrido), F# (2005 híbrido).

**Paradigma Lógico (1970s)**

En el paradigma lógico, un programa se expresa como un conjunto de hechos (declaraciones que son verdaderas sobre el dominio del problema) y reglas (relaciones lógicas entre esos hechos) escritas con fórmulas lógicas de primer orden.

La lógica de primer orden que admite el paradigma se limita a las Cláusulas de Horn, un subconjunto restringido de la lógica de primer orden, evitando así enfrentar el "problema de decisión" planteado por Hilbert el cual hacía referencia a toda la lógica de primer orden y que Turing y Church demostraron su indecibilidad.

La lógica matemática resulta una manera muy práctica para el humano de expresar formalmente problemas complejos y resolverlos mediante la aplicación de hechos y reglas. Por este motivo, la programación lógica ha resultado atractiva en diversos campos donde la programación imperativa no resulta la más adecuada.

Un lenguaje de programación lógica puro no tiene necesidad de estructuras de control como bucles o selección, el control de la ejecución es suministrado por el sistema subyacente.

El paradigma lógico encuentra su campo de aplicación más amplio en el desarrollo de sistemas de inteligencia artificial y algunos otros:

- **Sistemas basados en conocimiento**, en donde un sistema informático trata de imitar las recomendaciones de algún humano experto sobre algún dominio de conocimiento.
- **Demostración automática de teoremas**, en los cuáles, el programa genera nuevos teoremas a partir de una teoría existente.
- **Reconocimiento del lenguaje natural**, en donde los programas son capaces de comprender lo que un humano está tratando de expresar.

Pros:

- Facilidad para la IA y el procesamiento de lenguaje.
- Mecanismo de Búsqueda Integrado, no se requiere programar algoritmos de búsqueda complejos para problemas de satisfacción de restricciones.
- Expresividad, ideal para representar conocimiento y resolver problemas que involucran relaciones lógicas complejas, razonamiento simbólico y búsqueda de soluciones.
- Gestión de relaciones no direccionales, una relación puede funcionar en múltiples direcciones. Por ejemplo, padre(juan, X) busca a los hijos, mientras que padre(X, maria) busca al padre.
- Base matemática sólida, fundamentado en lógica matemática formal, permitiendo razonamientos rigurosos y verificación formal.

Contras:

- Curva de aprendizaje pronunciada, requiere un cambio de mentalidad significativo para programadores acostumbrados a paradigmas imperativos.
- Rendimiento limitado, generalmente más lento que lenguajes imperativos para tareas computacionales convencionales debido al overhead del motor de inferencia.
- Control limitado sobre la ejecución, se tiene menos control sobre el orden de evaluación y la eficiencia del algoritmo de búsqueda.
- Dificultad de depuración, es complejo replicar la lógica de resolución implementada por el motor de búsqueda y tener en cuenta los Cuts (!) si es que fueron implementados.
- Problemas de escalabilidad, puede tener dificultades con grandes volúmenes de datos o espacios de búsqueda muy extensos.

Algunos lenguajes que permiten aplicar el paradigma: Prolog (1972), Datalog (1977), Mercury (1995).

**Paradigma Descriptivo**

El paradigma descriptivo se centra en especificar cómo debe verse, organizarse o representarse la información. Se enfocan en la declaración de estructuras, formatos y metadatos, siendo interpretados por otros sistemas (navegadores, procesadores, motores de renderizado, gestores de base de datos) que se encargan de la

presentación o procesamiento final. Este subparadigma es fundamental en el desarrollo web, documentación técnica, intercambio de datos y configuración de sistemas.

Algunos lenguajes que permiten aplicar el paradigma: SQL (de consulta), HTML (de marcado), XML (de marcado), CSS (de formato), Json (de intercambio de datos).

## Lenguajes Puros vs Híbridos

Los lenguajes de programación conforman un universo de herramientas extremadamente amplio y variado. Esta heterogeneidad de lenguajes, permitirá clasificarlos según una gran variedad de aspectos. Específicamente, con respecto a los paradigmas de programación, los lenguajes pueden ser clasificados en híbridos o puros en función de aquellos paradigmas que permitan implementar.
Se consideran puros aquellos que implementan los principios fundamentales de un paradigma determinado sin combinarlo con conceptos asociados a otros paradigmas, respetando la coherencia y las reglas propias del paradigma en cuestión.
Se consideran híbridos o multiparadigma aquellos que permiten combinar múltiples paradigmas. Brindan mayor versatilidad al momento de desarrollar, al aceptar incorporar diferentes conceptos y herramientas propias de los paradigmas habilitados.
Los lenguajes utilizados en el ambiente profesional, en su gran mayoría, tienden a ser híbridos por la productividad y flexibilidad que otorgan, sin embargo, algunos desarrolladores frente a determinadas situaciones o desafíos puntuales, prefieren lenguajes puros por permitir mantener una línea de razonamiento más clara y predecible.
En el ambiente académico, se recurre a lenguajes puros cuando se tiene por objetivo incentivar en el alumno el razonamiento profundo para lograr construir nuevas estructuras mentales, impulsando el pensamiento crítico y su capacidad de abstracción al encontrarse “desafiado” por las reglas del paradigma.
La cátedra propone como lenguaje multiparadigma a C#, el cual posibilitará implementar el paradigma orientado a objetos, orientado a eventos y conceptos elementales de programación funcional.
Con miras a cátedras de años superiores, su versatilidad permitirá abordar conceptos más profundos (desarrollo de web apis, implementación de patrones de diseño, desarrollo de worker services, etc.) cuyo entendimiento se verá beneficiado al haber tratado con el lenguaje y su entorno en Paradigmas de Programación.
Con respecto al paradigma funcional se propone a Haskell, y para afrontar el paradigma lógico se propone a Prolog. Ambos lenguajes son clasificados puros, esta pureza permitirá explorar los conceptos asociados a ambos paradigmas de manera clara sin perder profundidad ni coherencia conceptual.